第一节：PHP 入门中文讲解

对应 PHP Crash Course.ppt，共 22 页。学习目标是看懂代码、预测结果，并解释 PHP 与 C/C++ 的相似处和差异。

1. PHP 是什么？（第 2 – 6 页）

PHP 是一种常用于服务器端的脚本语言，可以根据程序逻辑、用户输入和数据库内容生成 HTML。

典型流程：浏览器发请求 → 服务器执行 PHP → PHP 产生输出 → 浏览器接收 HTML。浏览器通常拿到的是输出，而不是 PHP 源码。PHP 也可以在命令行中运行，并不必须配合网页。

```
Hello
<?php
echo "World";
?>
```

PHP 标签外的 Hello 直接进入输出；标签内的 echo 执行后输出 World。这正是 intro.php 开头的原理。

普通语句通常以 ; 结束；if、for 等块结构不需要在每个结束花括号后添加分号。注释可以用 //、/* ... */，PHP 也支持 #。

课件把 PHP 称为 interpreted language，可按 “ 交给 PHP 引擎执行的脚本语言 ” 理解。现代实现会编译成操作码，还可能使用缓存或 JIT；不要理解成永远逐字符直接解释。

课件表述修正： PHP 通常放在 .php 文件里，但不是所有运行方式都要求该扩展名，例如 php -r 可执行直接传入的代码。PHP 支持成熟的面向对象编程，课件 “ 通常不是面向对象 ” 的说法不是语言限制。

2. 变量、类型和转换（第 7 – 9 页）

变量名以 $ 开头，不需要像 C++ 那样先写类型声明：

```
$x = 5; // 此时是整数
$x = "hello"; // 同一变量随后可以保存字符串
```

理解两个不同的词：

- 动态类型（dynamic typing）：变量的类型不必在编译时固定，运行中的值有类型。
- 弱类型/隐式转换（weak typing/coercion）：某些操作会自动转换操作数的类型。

PHP 不是 “ 没有类型 ” 。课件列出的 integer、double/float、string、boolean、array、object 都有实际意义，另外还有 null 等。

```
echo 5 + 5; // 10
```

```
echo 5 / 2; // 2.5
echo "5" + "10"; // 15：数值字符串参加算术
echo "5" . "10"; // 510：字符串连接
```

易错点： + 是算术加法，. 才是字符串连接。现代 PHP 不会把任意非数字字符串都安全地当作数字；例如 "abc" + 1 在 PHP 8 中会产生 TypeError。

== 与 ===（第 8 页）

```
5 == "5" // true：宽松比较允许类型转换
5 === "5" // false：值和类型都必须相同
5 === 5 // true
```

考试遇到比较题，要先写出两边的类型，再判断比较规则。不同 PHP 版本的宽松比较有差异，尤其涉及非数字字符串时；不要把“ 弱类型 ”当成万能转换规则。

3. 字符串与变量变量（第 9 页；结合演示代码）

双引号会进行变量插值，并解释 \n 等转义：

```
$fname = "Daniel";
$lname = "Katz";
echo "$fname $lname\n"; // Daniel Katz，随后换行
echo $fname . " " . $lname . "\n"; // 相同输出
```

单引号通常不插值，也不把 \n 当作换行；单引号字符串中主要特别处理 \\ 和 \'。

```
echo 'The cost is: $10' . "\n";
echo "The cost is: \$10\n";
```

两句都输出字面量 $10，后接换行。第二句中的 \$ 明确要求输出美元符号。

变量变量（variable variables）：把一个变量保存的字符串当作另一个变量的名字。

```
$fname = "Daniel";
$varname = "fname";
echo $$varname; // 等价于访问 $fname，输出 Daniel
```

推理顺序：$varname → "fname" → 访问 $fname → 得到 "Daniel"。它不是“ 双重美元输出 ”，也不是取地址。

4. 作用域（第 10、12 页）

在 PHP 中，函数里的普通局部作用域与外面的全局作用域不同。全局变量不会自动进入函数。

```
$x = 10;
function showX() {
    global $x;
    echo $x;
```

```
}
```

global $x 让函数内的 $x 绑定到全局的 $x。另外可以显式使用 $GLOBALS['x']。$_GET、$_POST、$_SERVER 等是 superglobal，在不同作用域中都可以访问。

```
function add($a, $b) {
        return $a + $b;
}
echo add(2, 3); // 5
```

这里 $a 和 $b 是函数参数，也是局部变量。return 把值交给调用者；echo 直接产生输出，二者不要混淆。

补充：普通参数默认按值传递；需要引用传递时显式写 &。这不意味着对象会按值深拷贝。

5. PHP 数组：有序映射（第 11 页）

不要只把 PHP array 想成 C 的“连续内存数组”。它更适合被理解为保持插入顺序的键 → 值映射，整数键和字符串键可以混用。

```
$a = array(100, 200); // 键 0、1
$b = array("name" => "Daniel", "age" => 20);
// 现代简写：$b = ["name" => "Daniel", "age" => 20];
```

```
foreach ($b as $key => $value) {
        echo "$key = $value\n";
}
```

foreach 直接遍历现有元素，适用于关联数组和不连续的整数键。

重要键转换：

- 整数键 2 就是整数。
- 可按 PHP 规则转换的十进制整数字符串 "2" 通常转换为整数键 2。
- 字符串 "2.5" 不是整数字符串，因此保持字符串键。
- 浮点键 2.5 则会转为整数键 2；在部分现代情形下还会报告精度丢失的弃用提示。

所以 "2.5" 与 2.5 当作数组键时不同！count($a) 是元素数，不是最大键加一。老师的 intro.php 正是在展示这一点。

6. 控制流（第 3 页）

课件强调 PHP 与 C/C++ 的 if、switch、while、do ... while、for 相似。以下是补充示例：

```
if ($score >= 90) {
        echo "A";
} elseif ($score >= 80) {
```

```
        echo "B";
} else {
        echo "Other";
}

for ($i = 0; $i < 3; $i++) {
        echo $i; // 连续输出 012
}
```

while 先判断条件；do ... while 先执行一次再判断，所以后者至少执行一次。for 的执行顺序是初始化 → 判断 → 循环体 → 更新 → 再判断。

大小写修正： PHP 变量名区分大小写，例如 $name 与 $Name 不同；函数名和类名通常不区分大小写，所以课件 “ PHP 全部大小写敏感 ” 的说法过于笼统。写代码时仍应保持拼写一致。

7. 类和对象（第 13 – 15 页）

```
class Thing {
        public $item;
        public function __construct($item) {
                $this->item = $item;
        }
}
$t1 = new Thing("book");
echo $t1->item;
```

- new 创建对象；-> 访问对象的属性或方法。
- :: 是作用域解析操作符，可用于类常量、静态成员及 parent::method() 等上下文。
- public：外部可访问；protected：类及相关继承范围内可访问；private：定义它的类内部可访问。
- extends 表示继承；parent::method() 调用父类实现。
- final 类不能被继承；final 方法不能被子类重写。它不是 “ 执行一次重写 ” 的关键字。
- __construct 是构造方法，__destruct 是析构方法。

对象比较也要区分：

```
$a = new Thing("book");
$b = new Thing("book");
$c = $a;
$a == $b;          // true：同一类，属性按宽松规则比较相等
$a === $b;         // false：两个不同实例
$a === $c;         // true：同一个实例
```

课件第 14 页的 T1 = new Thing(); 漏写 $；合法写法应是 $T1 = new Thing();（这里按课件无参数构造版本）。

8. 数据库与预处理语句（第 16 – 18 页）

标准思路：连接数据库 → 发出查询 → 获取结果 → 使用字段。

```
$myDB = mysqli_connect($dbserver, $dbuser, $dbpass, $dbName);
$result = $myDB->query("SELECT * FROM SomeTable");
$row = $result->fetch_object();
echo $row->MyAttribute1;
```

这是理解 API 流程的示意，假设连接和查询均成功且存在结果。查询多行时应循环获取，真实应用还要处理错误。

Prepared statement（预处理语句）把 SQL 结构与参数值分开，防止把用户提供的值解释成 SQL 代码。

```
$stmt = $myDB->prepare("SELECT * FROM SomeTable WHERE name = ?");
$name = "Daniel";
$stmt->bind_param("s", $name);
$stmt->execute();
$result = $stmt->get_result();
```

? 是参数占位符，"s" 表示绑定字符串。课件直接写 bind_param("s", "Daniel") 不合适，因为绑定的参数要求变量引用；应先放入 $name。课件 execute() 后也漏了分号。

第 17 页文字提到旧 mysql_query，但示例用的是 mysqli，不要混用；旧 mysql_* 扩展在现代 PHP 中已移除。get_result() 依赖 mysqlnd。

复习时能说明为什么字符串拼接 SQL 容易引入注入，以及参数化如何隔离值即可。占位符通常用于值，不能直接拿来替代表名、列名等标识符。

9. 字符串、文件和加载代码（第 19 – 21 页）

功能	常见函数	记忆例子
拆分字符串	explode	explode(",", "a,b") → ["a", "b"]
去掉右侧尾部字符	chop / rtrim	默认可去掉右侧空白
长度	strlen	返回字节数；不是通用 Unicode 字符数
比较	strcmp	相等返回 0；小于/大于按结果符号判断
查找子串	strstr	找到时返回从匹配处开始的子串，否则 false
大小写	strtoupper / strtolower	转换字母大小写

功能	常见函数	记忆例子
整文件读取/写入	file_get_contents / file_put_contents	简单文件读写
流式文件读写	fopen / fread / fclose	适合分块读取等场景

课件还列出 sha1、md5、addslashes、stripslashes、money_format：

- SHA-1/MD5 是散列函数，不是可逆加密；不要把它们当作现代密码存储的合适方案。
- addslashes 不是可靠的通用 SQL 注入防护；数据库例题应掌握参数化查询。
- money_format 在 PHP 8 中已移除；它是历史 API。

include 与 require 都会加载并执行另一文件中的代码。文件加载失败时，include 通常产生警告并允许继续；require 导致严重失败（现代 PHP 中是 Error，若不处理则终止）。include_once 和 require_once 用于避免重复加载。

10. exit 与 die（第 22 页：关键纠错）

课件写“ exit 结束这个脚本，die 结束所有脚本 ”，这一区别不成立。

die() 是 exit() 的别名，二者都会终止当前 PHP 执行。 即使在被 include 的文件里调用，也会终止当前请求/执行流程，而不是只返回到外层文件。若想从被包含文件返回，应理解 return 的作用。

复习检查

你应能不看答案说明：. 与 +、单双引号、== 与 ===、$x 与 $$x、局部与全局作用域、数组键与元素数、-> 与 ::、对象属性相等与同一实例、参数化 SQL、include 与 require，以及 exit 与 die 的关系。

第二节：描述语法与语义

对应 pl12ch3.ppt 第 3 章，共 57 页。本章的主线是：语法描述程序的结构，语义描述程序的含义。能写出程序，并不代表已经精确说明程序如何执行。

1. 基础术语（第 3 – 5 页）

英文	中文与含义	例子
Syntax	语法：表达式、语句、程序的形式或结构	赋值语句有哪些组成部分？
Semantics	语义：这些结构的含义	赋值后哪个变量变成什么值？
Sentence	句子：语言中一个完整的合法符号串；课件也先泛指字母表上的串	文法生成的一条赋值语句
Language	语言：一组句子的集合	所有符合文法的程序
Lexeme	词素：代码里实际出现的最小词法单位	sum、+、123
Token	词法类别	identifier、plus、integer_literal

例如 sum = sum + 10 中，两处 sum 都是 lexeme，所属 token 类别都是 identifier。10 是具体文本，其类别是 integer_literal。不要把“名字本身”与“名字类别”混淆。

Recognizer（识别器）读入一个串，回答它是否属于语言。编译器的语法分析器是一个例子。

Generator（生成器）通过规则产生语言中的串。文法就是一种生成方式。它不等于“生成机器代码的编译器后端”。

2. BNF 和上下文无关文法（第 6 – 13 页）

BNF（Backus – Naur Form）是表示上下文无关文法（CFG）的记法；两者具有对应的描述能力。

```
<stmt>  →  <var>  =  <expr>
<var>    →  a  |  b  |  c
<expr>  →  <var>  +  <var>  |  <var>
```

- <stmt>、<var>、<expr> 是非终结符（nonterminals），表示还可展开的语法类别。
- a、b、c、=、+ 是终结符（terminals），在当前文法中不能继续展开。
- → 表示产生式；左边 LHS 是一个非终结符，右边 RHS 是终结符/非终结符组成的串。
- | 表示选择，不是最终代码里必须出现的竖线。
- 开始符号（start symbol）是推导的入口。

“上下文无关”的重点是左边只有一个非终结符，展开它不依赖它周围的符号。文法常记为 G = (S, N, T, P)：开始符号、非终结符集合、终结符集合、产生式集合。

课件第 7 页把 nonterminal 顺带称为 terminal，属于文字错误；这两个术语必须分清。

用递归描述列表（第 10 页）

```
<ident_list>  →  ident  |  ident  ,  <ident_list>
```

可以生成一个 identifier，也可以生成 identifier, identifier, identifier 等任意有限长度的非空列表。ident 在这里代表 token 类别；实际输入可以是 a,b,c。

递归分支使列表能继续延长，非递归分支使推导能结束。如果只有递归而没有结束分支，就无法生成有限的全终结符句子。

推导题怎么做？（第 11 – 13 页）

课件的文法：

```
<program>  →  <stmts>
<stmts>    →  <stmt>  |  <stmt>  ;  <stmts>
<stmt>     →  <var>  =  <expr>
<var>      →  a  |  b  |  c  |  d
<expr>     →  <term>  +  <term>  |  <term>  -  <term>
<term>     →  <var>  |  const
```

生成 a = b + const 的最左推导：

```
<program>
⇒  <stmts>
⇒  <stmt>
⇒  <var>  =  <expr>
⇒  a  =  <expr>
⇒  a  =  <term>  +  <term>
⇒  a  =  <var>  +  <term>
⇒  a  =  b  +  <term>
⇒  a  =  b  +  const
```

每一步只替换一个非终结符；最左推导每次选最左的那个。最右推导每次选最右的那个。中间的任何形式都叫 sentential form（句型）；全是终结符的句型才是该文法的句子。

注意按给定规则作答：这个课件文法的 <expr> 必须包含一次 + 或 -，因此不能生成单独的 a = b。也不能擅自补上乘法、括号或更长的表达式。

3. 语法树、歧义、优先级和结合性（第 14 – 18 页）

Parse tree（语法分析树）把推导画成树：根是开始符号，内部节点是非终结符，其子节点是所使用产生式的右侧；叶子从左到右读出句子。

上面的赋值可以简写为：

```
<stmt>
├──── <var> → a
├──── =
└──── <expr>
      ├──── <term> → <var> → b
      ├──── +
      └──── <term> → const
```

歧义文法（ambiguous grammar）：存在一个句子，可以得到两棵不同的语法分析树。证明一个文法歧义，只需给出一个这样的反例。

```
<expr> → <expr> <op> <expr> | const
<op>   → / | -
```

这个文法没有区分 / 与 - 的层级。例如把三个 const 具体理解成 8、4、2，则 8 - 4 / 2 有两种结构：

```
(8 - 4) / 2   → 2
8 - (4 / 2)   → 6
```

证明歧义的关键是两种树结构，不是两种数值；有时两棵树恰好算出相同结果，文法仍然歧义。

两个推导顺序不必然意味着歧义。同一棵树可以有不同的展开顺序。想用推导证明歧义，应给出两个不同的最左推导，或直接给出两棵树。

优先级：用文法层级表达

```
<expr> → <expr> - <term> | <term>
<term> → <term> / const | const
```

除法在 <term> 层内完成，减法在 <expr> 层组合，所以除法绑定得更紧。现在 8 - 4 / 2 只能按 8 - (4 / 2) 解释。

结合性：同优先级时如何分组

```
<expr> → <expr> - const | const      // 左结合
```

8 - 3 - 1 的结构是 (8 - 3) - 1 = 4。

补充右结合例子：

```
<expr> → const - <expr> | const      // 右结合
```

现在同一个串结构是 8 - (3 - 1) = 6。

优先级解决不同运算符谁先组合，结合性解决同级运算符向哪边组合。课件第 18 页用加法示范左递归；学习时用减法更容易看出分组对结果的影响。

4. EBNF：简化 BNF 的书写（第 19 – 21 页）

记号	含义	例子		
[...]	可选，0 次或 1 次	ident [(<expr_list>)]		
{ ... }	重复，0 次或更多次	`letter { letter	digit }`		
`(a	b)`	分组选择	`(+	-)`

这些是元语言记号，不一定是被描述语言里的实际字符。阅读具体文法时要区分作为终结符的括号与用于分组的括号。

BNF：

```
<expr> → <expr> + <term> | <expr> - <term> | <term>
```

对应的 EBNF：

```
<expr> → <term> { (+ | -) <term> }
```

可以出现一个 term，后面跟零个或多个“ 运算符 + term ”。EBNF 更简洁，但没有因此获得超出 CFG 的语言生成能力。

补充：EBNF 的重复结构本身给出序列形式；实现其计算时仍需明确如何折叠这个序列。课件中的算术例子通常按对应 BNF 的左结合来解释。

5. 静态语义与属性文法（第 22 – 29 页）

CFG 善于检查形式，但一些约束不能方便地只靠 CFG 表达，例如变量是否已声明、运算数类型是否满足要求、某个名字绑定到哪一项声明。

这些常归入 static semantics（静态语义）：不用真正执行程序，就能分析的上下文约束。课件说“ Nothing to do with meaning ”，可以理解成它不是在描述运行过程，不能理解成类型和名字规则完全不属于语义。

Attribute grammar（属性文法）= CFG + 节点属性 + 属性计算规则 + 检查谓词。

给语法树节点附上类型等信息，这叫给树做 decoration（属性标注）。

三种属性要分清

- Synthesized attribute（综合属性）：通常从子节点的信息算出父节点的信息，向上汇总。例如由两个操作数的类型得到表达式类型。
- Inherited attribute（继承属性）：由父节点或兄弟节点等上下文提供，向下或横向传递。例如赋值目标把预期类型传给右侧表达式。
- Intrinsic attribute（固有属性）：一开始就提供的信息，例如标识符文本、字面量值；标识符类型通常通过符号表查询得到。

用课件的类型例子走一遍

```
<assign>  →  <var>  =  <expr>
<expr>      →  <var>[1]  +  <var>[2]  |  <var>
<var>         →  A  |  B  |  C
```

[1]、[2] 在这里标记两个节点的位置，不是程序中的数组下标。

对于 A = B + C，简化教学规则如下：

```
expr.expected_type  ←  左侧  A  的  actual_type
var[1].actual_type     ←  lookup(B)
var[2].actual_type     ←  lookup(C)
expr.actual_type       ←  var[1].actual_type

检查：var[1].actual_type  ==  var[2].actual_type
检查：expr.expected_type  ==  expr.actual_type
```

若 A、B、C 都是 int，两个检查都通过。若 B 是 int、C 是 real，第一个失败；若 B、C 都是 int 但 A 是 real，第二个失败。

这是课件选择的一套严格示例规则。实际语言可能允许数值提升或赋值转换，需要另外制定属性规则；不能把示例当成所有语言的标准行为。

纯综合属性通常能自底向上求值；纯继承属性在课件描述的简单依赖情形可自顶向下求值。混合时要遵守依赖顺序：先有计算所需的信息，再求该属性。补充：兄弟间依赖也会限制访问顺序，存在循环依赖时不能仅凭一次遍历完成。

6. 为什么需要动态语义？（第 30 页）

形式正确并不足以告诉我们如何执行程序。语言使用者、编译器实现者和程序证明者需要知道：表达式的值如何求、赋值如何改变状态、循环何时继续。

本章主要介绍三种方法：

方法	核心问题	主要工具
Operational semantics	程序一步步如何运行？	抽象机器、执行规则、状态迁移
Denotational semantics	程序对应什么数学对象或函数？	递归定义、状态映射
Axiomatic semantics	执行前后哪些性质成立？	断言、Hoare 三元组、证明规则

7. 操作语义 Operational Semantics（第 31 – 34 页）

通过在实际或理想化机器上执行语句，用机器状态变化解释语句的含义。

补充例子：初始状态 x = 2，执行 x = x + 3：

```
读  x  →  得到  2
```

```
计算 2 + 3 → 得到 5
把 5 写入 x → 新状态 x = 5
```

为了不依赖某一真实 CPU，正式描述通常使用抽象机器。课件的理想化机器方案包含翻译器和模拟器。

- Natural operational semantics 通常又叫 big-step：从一个程序及初始状态直接描述到最终结果。
- Structural operational semantics 通常用 small-step：描述每一个小执行步骤如何迁移。

这些术语的细化是帮助理解的补充。课件的评价是：非正式解释直观，完全正式地覆盖一门大型语言则可能非常复杂。

8. 指称语义 Denotational Semantics（第 35 – 44 页）

把语言结构映射成数学对象，例如把表达式映射成 “ 给定状态后得到一个值 ” 的函数，把语句映射成 “ 把一个状态变成另一个状态 ” 的函数。

状态与取值

```
s = { 〈x, 2〉, 〈y, 7〉 }
VARMAP(x, s) = 2
VARMAP(y, s) = 7
```

状态 s 保存所有当前变量及其值。这里 VARMAP 是数学查询函数，不是 PHP 内置函数。

数字的含义（第 38 页）

```
Mdec("7") = 7
Mdec(已有数字串后接数字 d) = 10 × Mdec(已有数字串) + d
```

因此 Mdec("123") = 10 × (10 × 1 + 2) + 3 = 123。这是把文本串映射成整数，而不是说字符本身就是整数。

表达式映射（第 39 – 40 页）

可写成 Me(E, s)：表达式 E 在状态 s 中的值。课件简化模型把表达式映射到整数集合 Z 或 error。

```
Me(常量, s) = 常量的数值
Me(x, s) = VARMAP(x, s)，若变量未定义则 error
Me(E1 + E2, s) = Me(E1, s) + Me(E2, s)
```

若子表达式产生错误，整体也应传播错误。课件第 40 页对 undef/error 的写法不完全一致，学习时应区分 “ 查询得到未定义 ” 与 “ 整个求值函数返回错误 ”，并统一错误传播规则。

例：在 {x=2,y=7} 中，Me(x + y, s) = 9。

赋值映射（第 41 页）

Ma(x := E, s) 先在旧状态中求 Me(E,s)，然后只更新 x，其他变量不变。

```
s = {x=2, y=7}
Ma(x := x+y, s) = {x=9, y=7}
```

这不是把 x、y 都改为 9。符号 := 用来强调赋值；它不表示数学上的恒等关系。

循环映射（第 42 – 43 页）

```
Ml(while B do L, s) =
        条件求值出错：error
        条件为 false：s
        条件为 true：先执行 L 得到 s1，再求 Ml(同一个循环, s1)
```

循环的迭代因此被写成状态函数的递归。

补充手算：while x < 3 do x := x+1，初始 x=0：

```
{x=0} → {x=1} → {x=2} → {x=3}
```

在 x=3 时条件为假，返回状态 {x=3}。如果循环永不结束，就不能当成返回了某个正常最终状态；完整理论需要另外处理不终止（例如部分函数或底元素）。课件主要展示正常结束和错误的情况。

与操作语义的差别是描述方式：操作语义给执行步骤/算法；指称语义给严谨的数学映射。二者都可以描述状态变化，不是“ 操作语义有状态、指称语义没状态 ”。

9. 公理语义 Axiomatic Semantics（第 45 – 55 页）

用逻辑断言描述执行前后的性质，用于证明程序正确。

```
{P} S {Q}
```

P 是前置条件（precondition），S 是语句，Q 是后置条件（postcondition）。常见的部分正确性解释是：若开始时 P 成立，而且 S 正常终止，则结束时 Q 成立。

最弱前置条件 Weakest Precondition

它是保证指定后置条件的限制最少的前置条件。

课件例子：

```
a := b + 1
后置条件：a > 1
```

将后置条件中的 a 替换成 b+1，得到 b+1 > 1，化简为 b > 0。b > 10 也能保证结果，但太强；b > -1 又可能允许结果不满足要求，所以太弱。

赋值规则：代换（第 49 页）

```
{Q[x ← E]} x := E {Q}
```

$Q[x \leftarrow E]$ 表示把 Q 中 x 的自由出现替换成 E，避免变量捕获；在本章简单算术例子中直接代换即可。不是替换整个程序，也不是反过来把 E 换成 x。

补充例子：

```
x := x + 2
Q：x > 5
wp：x + 2 > 5，即 x > 3
```

假设采用无溢出的数学整数模型；实际语言的溢出、异常等行为可能需要额外条件。

顺序语句：从后往前算（第 50 页）

```
若 {P1} S1 {P2}，且 {P2} S2 {P3}
则 {P1} S1; S2 {P3}
```

例：

```
x := x + 1;
y := 2 * x;
Q：y > 10
```

先处理后一句：$2*x > 10$，即 $x > 5$。

再处理前一句：$x+1 > 5$，即初始 $x > 4$。

这是反向推导逻辑条件，不是程序按倒序执行。

条件语句（第 51 页）

同一个前置条件 P，在真分支与假分支都必须足以保证 Q：

```
{P ∧ B} S1 {Q}
{P ∧ ¬B} S2 {Q}
————————————————————————————————
{P} if B then S1 else S2 {Q}
```

补充例子：if x >= 0 then y := x else y := -x，后置条件 $y >= 0$。两个分支分别利用 $x>=0$ 与 $x<0$，前置条件可以为 true（数学整数，无溢出）。

后果规则 Rule of Consequence（第 49 页）

已知 $\{P\}\ S\ \{Q\}$，如果 $P' \Rightarrow P$ 且 $Q \Rightarrow Q'$，就可得 $\{P'\}\ S\ \{Q'\}$。

通俗地说：前置条件可以加强，后置条件可以减弱。若原来在 $x>0$ 时保证 $y>1$，那在更严格的 $x>10$ 时当然也保证较弱的 $y>0$。

循环不变式 Loop Invariant（第 52 – 54 页）

不变式 I 是每次检查循环条件时都成立的性质。它不是“ 某个变量永远不变 ”，而是某个关系保持为真。

证明框架：

1. 初始化：$P \Rightarrow I$，开始时成立。

2. 保持性：$\{I \wedge B\}\ S\ \{I\}$，在条件为真时执行循环体，仍保持 I。

3. 退出：$I \wedge \neg B \Rightarrow Q$，退出时能推出目标。

4. 若要证明完全正确性，还要证明循环终止。

课件还要求条件求值不破坏 I。一般教学例题假设 B 无副作用。

补充完整例子：

```
前提：n  是非负整数
i  :=  0;
while  i  <  n  do
        i  :=  i  +  1
目标：i  =  n
```

选择 I：0 <= i <= n（同时保留 n 是非负整数且循环不改变 n）：

- 初始 i=0，所以 I 成立。
- 若 I 成立且 i<n，整数性保证加 1 后仍有 i<=n，且不小于 0。
- 退出时 i>=n；结合不变式 i<=n，得 i=n。
- 终止量 n-i 在每次执行循环体时是正整数，之后严格减少，因此不能无限执行。

为什么不直接选择 i=n 作不变式？因为当 n>0 时初始 i=0 不满足它。不变式需要能在循环前成立，也要配合退出条件推出最终结果。

部分正确性与完全正确性： 前者证明“ 如果终止则结果正确 ”；后者还证明一定终止。不应把不变式证明自动当成终止证明。

10. 考前抓住什么？（第 56 – 57 页）

建议优先掌握以下能力：

1. 区分 lexeme/token、terminal/nonterminal、句型/句子、syntax/semantics。

2. 按给定文法写最左或最右推导，画出树。

3. 用两棵树证明歧义；用层级和递归方向表达优先级、结合性。

4. 解释 EBNF 的可选、重复、选择，转换简单 BNF/EBNF。

5. 追踪属性依赖，区分 actual_type 的综合属性和 expected_type 的继承属性。

6. 用同一段程序说明三种语义方法的差别。

7. 求赋值与顺序语句的最弱前置条件，检查循环不变式和终止量。

把 x := x+1 放在一起看：操作语义说“ 读取、相加、写回 ”；指称语义说“ 映射到只把 x 更新为旧值加 1 的状态 ”；公理语义说“ 为保证执行后 x>5，执行前应有 x>4 ”。这就是三种方法最实用的区别。

intro.php：老师演示代码逐段讲解

原件：intro.php。原始文件只有 42 行；以下按它原本的内容讲解，不修改演示代码。

先分清正在执行的代码与注释掉的例子。 当前有效部分是开头的普通文本、数组赋值和 foreach。姓名、字符串、变量变量及算术例子都在 /* ... */ 中，不会执行。

1. PHP 标签外的文本

```
Hello  World
<?php
```

Hello World 在 PHP 开始标签之前，因此直接输出。原文件紧随其后的行结束符也会输出。它不是 echo 语句，不需要分号。

<?php 之后进入 PHP 代码；末尾 ?> 结束 PHP 代码区。纯 PHP 文件中通常省略结束标签，避免意外空白输出，但老师的文件是为了演示标签内外的区别。

2. 创建数组与键的类型

```
$arr  =  array();
$arr[0]  =  100;
$arr[1]  =  200;
$arr["2.5"]  =  10;
$arr[3]  =  400;
```

逐行执行后的数组内容：

插入顺序	键	键的类型	值
1	0	integer	100
2	1	integer	200
3	"2.5"	string	10
4	3	integer	400

所以 count($arr) = 4。但这不是“ 下标 0、1、2、3 的四个连续元素 ”：整数键 2 不存在。

"2.5" 是带引号的字符串，不是浮点数。只有符合转换规则的整数字符串会转为整数键；"2.5" 不符合。

这段代码意在打破“ 数组总有连续整数下标 ”的直觉。数组元素数与键的取值是两件事。

3. 被注释的 for 循环为什么不适合这里？

```
/*for   ($i   =   0;   $i<count($arr);   $i++)
            echo   "arr[$i]   =   $arr[$i]\n";
*/
```

现在整个块都不会执行。假如单独取消这块注释，$i 会是 0、1、2、3，因为 count($arr) 是 4。

概念上的过程：

```
i=0：读取整数键   0，值   100
i=1：读取整数键   1，值   200
i=2：读取整数键   2，键不存在
i=3：读取整数键   3，值   400
```

在现代 PHP 中读取不存在的键一般产生 Undefined array key 2 警告，读取值通常为 null，插值输出为空；旧版本的诊断等级/措辞可能不同，是否显示也由错误报告设置决定。

这条循环根本没有读取字符串键 "2.5"，因此漏掉值 10。不是循环次数不够，而是遍历方式假设了不存在的键结构。

双引号里 arr[$i] 的第一部分只是普通文本，其中 $i 插值；右侧 $arr[$i] 才是数组元素访问。

4. 当前执行的 foreach

```
foreach($arr   as   $key=>$val){
            echo   "$key   =   $val\n";
}
```

foreach 依次拿出实际存在的元素，把键赋给 $key，把值赋给 $val。=> 是键和值的分隔符，不是“大于等于”操作符。

按插入顺序产生：

```
0   =   100
1   =   200
2.5   =   10
3   =   400
```

这里 2.5 输出时不带引号，但其底层键依然是 string。输出文本不会自动展示类型。

当前文件的整体预期可见输出

```
Hello   World
0   =   100
1   =   200
2.5   =   10
3   =   400
```

这是按代码语义推导的结果。原件开头使用 CRLF 行结束符，而 \n 产生 LF；上面只展示可见行内容，未声称逐字节一致。结束标签后的尾部行结束符按 PHP 的标签处理规则解释，不能随意

额外算作一条输出语句。

若通过浏览器看生成的普通 HTML，\n 通常不会呈现为独立换行，因为 HTML 会折叠普通空白；命令行、响应源码或预格式化文本才适合核对这些行。

5. 单独访问数组元素

```
//echo  $arr[3];
```

它目前是单行注释，不执行。若取消注释，则输出 400，没有自带换行。

6. 姓名与字符串拼接

以下属于原文件的一个被注释的大块。若单独启用这一块：

```
$fname  =  "Daniel";
$lname  =  "Katz";
echo  $fname."  ".$lname."\n";
echo  "$fname  $lname\n";
```

两行均输出：

```
Daniel  Katz
```

第一行用 . 拼接四个字符串部分；第二行在双引号内部插值。它们是两种不同语法，产生相同文本。

```
echo  "The  cost  is:  \$10\n";
echo  'The  cost  is:  $10'."\n";
```

两行均输出 The cost is: $10 后接换行。第一行在双引号中转义美元符号；第二行的单引号内容是字面文本，再拼接一个真正的换行。

```
//echo  "$fname  $name\n";
```

它不执行，而且 $name 没有在演示中定义。如果启用，可能产生未定义变量诊断；它不是 $lname 的别名。变量名必须一致。

7. 变量变量：两层查找

```
$varname  =  "fname";
echo  "The  $varname  is  ".$$varname."\n";
$varname  =  "lname";
echo  "The  $varname  is  ".$$varname."\n";
```

第一条输出的推理：

```
"The  $varname  is  "  →  "The  fname  is  "
$$varname  →  $fname  →  "Daniel"
```

第二条同理。若启用整个姓名块，合起来的输出为：

```
Daniel  Katz
Daniel  Katz
The  cost  is:  $10
The  cost  is:  $10
The  fname  is  Daniel
The  lname  is  Katz
```

`$varname` 保存变量名字；`$$varname` 访问那个名字对应的变量的值。没有改变 `$fname` 或 `$lname` 的内容。

8. 最后一块：算术、连接、隐式转换

```
/*echo  "Hello  world\n";
echo  5+5;
echo  "\n";
echo  5/2;
echo  "\n";
echo  "Daniel"  .  "Katz\n";
echo  "5"  +  "10";
echo  "\n";*/
```

这块当前不执行。若单独启用，预期输出：

```
Hello  world
10
2.5
DanielKatz
15
```

- `5+5` 是整数加法，得到 10。
- `5/2` 是除法，得到 2.5；不要套用 C/C++ 的两整数相除截断规则。
- `"Daniel" . "Katz\n"` 用 `.` 连接，没有添加空格，所以是 `DanielKatz`。
- `"5" + "10"` 是数值字符串做加法，得到 15，不是 `510`。
- 单独 `echo "\n"` 的用途是给前一句没有换行的输出补上换行。

9. 最值得掌握的六个结论

1. 标签外的文本直接输出，标签内的代码执行后产生输出。

2. `/* ... */` 和 `//` 中的演示现在不会运行；不要把它们全加进当前输出。

3. PHP array 是有序映射，元素数不保证整数键连续。

4. "2.5" 是字符串键；本例没有整数键 2。

5. 双引号可插值，. 连接字符串，+ 做数值运算。

6. $$varname 是按变量保存的名字再次查找变量。

10. 自己检查是否理解

先不运行，回答以下变化会怎样：

- 把 $arr["2.5"] 改成 $arr["2"]：键会变成整数 2，此时四个键正好连续。
- 在原数组后增加 $arr[] = 500：对这个具体数组，会增加整数键 4；字符串键 "2.5" 不决定下一个整数键。
- 把 "5" + "10" 改成 "5" . "10"：得到文本 510。
- 把姓名插值改成 echo '$fname $lname';：输出变量名的字面文本，不是姓名。

如果你能解释这些结果的原因，而不是只背输出，就已经掌握了演示的重点。

考前练习与答案

这些题根据课件自行编写，不是老师提供的真题。先做题再看答案；程序证明题采用数学整数，无溢出，条件无副作用。

A. PHP 与 intro.php

题 1：预测输出

```
$a = "5";
$b = 10;
echo $a + $b;
echo "\n";
echo $a . $b;
```

题 2：判断比较结果

```
5 == "5"
5 === "5"
```

说明原因，不能只回答 true/false。

题 3：数组的键

```
$a = [];
$a[0] = 10;
$a["2.5"] = 20;
$a[3] = 30;
```

问：count($a) 是多少？整数键 2 存在吗？for ($i=0; $i<count($a); $i++) 能完整遍历吗？

题 4：变量变量

```
$x = "score";
$score = 95;
echo "$x = " . $$x;
```

题 5：解释作用域

```
$n = 10;
function f() {
    echo $n;
}
```

为什么函数不能直接依靠外面的普通 $n？给出一种显式访问方式。

题 6：纠错

判断下面三句话是否正确，并修正：

1. die() 终止所有脚本，而 exit() 只退出当前被包含文件。

2. bind_param("s", "Daniel") 是课件数据库例子的合适写法。

3. addslashes 可以替代参数化 SQL。

题 7：对象比较

```
class Box { public $v = 1; }
$a = new Box();
$b = new Box();
$c = $a;
```

判断 $a == $b、$a === $b、$a === $c。

B. 文法与属性

题 8：术语

在 total = count + 12 中，指出一个 lexeme 及其 token 类别。为什么它们不是同一个概念？

题 9：最左推导

```
<S> → <V> = <E>
<V> → a | b
<E> → <V> + <V> | <V>
```

写出生成 a = b + a 的最左推导。

题 10：歧义

```
<E> → <E> - <E> | n
```

用 n - n - n 证明它歧义。两个展开顺序不同就一定是歧义吗？

题 11：优先级与结合性

```
<E> → <E> - <T> | <T>
<T> → <T> / n | n
```

解释 / 与 - 的优先级，以及每个运算符的结合性。把 n 具体化为数字后，12 - 6 / 3 如何分组？

题 12：EBNF 转 BNF

```
<L> → ident { , ident }
```

改写成只使用 BNF 的规则，并判断空列表是否属于语言。

题 13：属性文法

对于 A = B + C，给定 A:int、B:int、C:real，使用课件的严格同类型规则时能否通过？actual_type 与 expected_type 分别是什么属性？

C. 动态语义与证明

题 14：三种语义

分别用操作语义、指称语义和公理语义的思路解释 x := x+2。

题 15：状态更新

初始状态 {x=3, y=4}，执行 x := x+y，给出最终状态，并说明右边在哪个状态求值。

题 16：最弱前置条件

```
x := 3*y + 1
后置条件：x > 10
```

求最弱前置条件。y>100 是不是一个有效前置条件？是不是最弱的？

题 17：顺序语句

```
x := x + 2;
y := x - 1;
后置条件：y >= 5
```

求最弱前置条件。

题 18：循环不变式

```
前提：n 是非负整数
i := 0;
while i < n do i := i+1
后置条件：i = n
```

给出不变式、初始化与保持性说明、退出推论以及终止量。

题 19：部分正确性

“已经证明循环不变式，所以程序一定结束。”这句话哪里有问题？

答案与解题思路

1 – 7：PHP

1. 第一行 15，第二行 510。+ 将数值字符串用于算术，. 用于连接。

2. 分别 true、false。宽松比较允许数字字符串转换；严格比较要求类型也相同。

3. 元素数为 3，整数键 2 不存在。这个 for 访问 0、1、2，既会访问缺失键，也漏掉字符串键 "2.5" 和整数键 3。元素数不是遍历键的可靠上界。

4. 输出 score = 95。$x 得到名字 score，$$x 再访问 $score。

5. 函数局部作用域不会自动导入普通全局变量。可在函数内写 global $n;，或显式使用 $GLOBALS['n']；也可改为传参数。

6. 三句都不对。die 与 exit 等价；绑定参数应先用 $name = "Daniel"，再 bind_param("s", $name)；参数化 SQL 隔离 SQL 结构与输入值，addslashes 不是它的可靠替代。

7. true、false、true。第一项比较同类对象的属性，后两项判断是否同一实例。

8 – 13：文法

8. total 是 lexeme，类别是 identifier；12 是 lexeme，类别是 integer_literal。具体文本与抽象类别不同。

9. 推导：

```
<S>
⇒  <V>  =  <E>
⇒  a  =  <E>
⇒  a  =  <V>  +  <V>
⇒  a  =  b  +  <V>
⇒  a  =  b  +  a
```

10. 两种结构是 (n-n)-n 与 n-(n-n)，可分别形成不同的树。仅改变非终结符展开顺序，可能仍对应同一棵树，不能单凭此判断歧义。

11. / 优先于 -；两者在各自层级均由左递归表达左结合。例子分组为 12 - (6/3)，值为 10。

12. 一种写法：

```
<L>  →  ident  |  ident  ,  <L>
```

或显式使用空尾部：

```
<L>  →  ident  <Tail>
<Tail>  →  ,  ident  <Tail>  |  ε
```

ε 表示空串。尾部可以为空，但整个列表至少有一个 ident，因此不包含空列表。

13. 无法通过：B 与 C 的类型不同，违反操作数同类型谓词。actual_type 是综合属性，expected_type 是继承属性。期望类型 int 从赋值目标 A 传入；不能默许课件没有规定的类型提升。

14 – 19：语义与证明

14. 操作语义：读旧 x、加 2、写回。指称语义：状态映射 s s[x s(x)+2]。公理语义：若后置条件要求 x>5，可用 {x>3} x:=x+2 {x>5} 描述前后性质。

15. {x=7,y=4}。先在旧状态中求右侧 3+4，只更新 x。

16. 代换后得到 3*y+1>10，化简为 y>3；若强调 y 是整数，也可写 y>=4。y>100 有效但过强，不是最弱。

17. 先处理最后一句：x-1>=5，即 x>=6。再处理第一句：x+2>=6，所以初始 x>=4。

18. 不变式 0<=i<=n，同时 n 是固定非负整数。初始 i=0；若 i<n，则整数 i 加 1 仍不超过 n；退出时 i>=n，结合 i<=n 得 i=n。终止量 n-i 非负且每次执行循环体严格减少。

19. 不变式只证明保持性及退出时的性质，不保证一定退出。还要证明终止，才能从部分正确性提升到完全正确性。

最后一次口头复习

不看讲义，用自己的话讲清下面三句话：

- PHP 的元素个数为什么不等于可访问的连续下标数？
- 一门文法如何同时确定运算符优先级与结合性？
- 操作语义、指称语义、公理语义如何从三个角度描述同一条赋值？

能把原因说清，再把推导、输出和最弱前置条件写对，比只记英文定义更接近“弄懂”。